PORTFOLIO

100년 동안 꾸준히 성장하는 개발자, 한승규입니다.

Email

trust1204@gmail.com

Github

https://github.com/Seungkyu-Han

Phone

+82 10-4081-9491

Blog

https://seungkyu.tistory.com/
https://seungkyu-han.tistory.com/

INTRODUCTION

안녕하세요, 신입 백엔드 개발자 한승규입니다.

100년 동안 꾸준히 성장하는 개발자, 한승규입니다.

Kotlin, Spring Webflux, MSA, Kafka등의 최신 기술을 사용하여 개발한 경험이 있습니다.

알고리즘 및 컴퓨터과학을 적용하여 ‘서버의 처리량’을 높이는 것에 관심이 있습니다.

다양한 팀과의 협업을 중요하게 생각하며,
다른 분야의 의견을 경청하고 이를 개발 과정에 반영하는 데 적극적입니다

새로운 기술을 익히고 적용하는 것에 흥미가 있습니다.



EDUCATION

학력 사항

경북대학교 IT대학

컴퓨터학부 심화컴퓨터공학전공, 2019.03 - 2025.02 (석차 1/87)

150

수강학점

4.43

평균 학점(4.5 기준)

수상 및 활동

2023년도 경북대학교 컴퓨터학부 이공계 장학생 선정(과학기술정보통신부)

2023년도 경북대학교 IT대학 중앙동아리 연합 해커톤 우수상(경북대학교 IT대학)

2024년도 계명대학교 GenAI 해커톤(생성형 AI 해커톤 대회) 멘토

2024년도 IPESK 차세대 공학자 선정

WORK EXPERIENCE

경력 사항

NexaDS(넥사디에스)
하계 인턴, 2023.07-2023.08

경북대학교 현장실습 프로그램의 일환으로 NexaDS에서 8주간 인턴 진행
회사의 고객 관리 효율성을 높이기 위해 Spring Boot를 활용한 CRM 시스템 개발 프로젝트 참여

경북대학교 산학협력
교내 활동, 2023.09-2023.12

경북대학교 캡스톤 디자인 프로그램으로 NexaDS와 산학연계 프로젝트 진행
물류 시스템의 자재 관리 및 수주·발주 프로세스를 위한 WMS(창고관리시스템)를 개발

모키(키오스크 플랫폼)
하계 인턴, 2024.07-2024.08

경북대학교 현장실습 프로그램의 일환으로 모키에서 4주간 인턴 진행
관리자가 매출 현황을 모니터링하고, 인공지능을 통해 매출 예측을 모니터링하는 서비스를 개발

계명대학교 IT융합교육센터
프리랜서, 2024.04-2024.12

2000명이 수강하는 원격강의에서 학생들과 교수의 소통을 위해, 기존 LMS(학습관리 시스템)의
문제점을 분석하여 개선하고 챗봇과 채팅이 포함된 LMS(학습관리 시스템)을 개발

SKILLS

기술

프론트엔드

HTML5, CSS3, Javascript

백엔드

Java, Kotlin, Python

Spring Boot, Spring Webflux, Fast API

Spring Security, Spring Cloud, JPA, QueryDSL, R2DBC, Swagger, ReactiveX, JWT

MySQL, MS-SQL, MongoDB, Redis, Kafka, Debezium

JUnit5, Apache Jmeter, Locust

Devops

AWS(EC2, RDS, S3, Route53), GCP

Docker, Docker-compose, Jenkins

PROJECTS

크무톡톡

Toge-do

소리로 일기

MSA 프로젝트

최근에 수행한 프로젝트 위주로 작성했습니다.

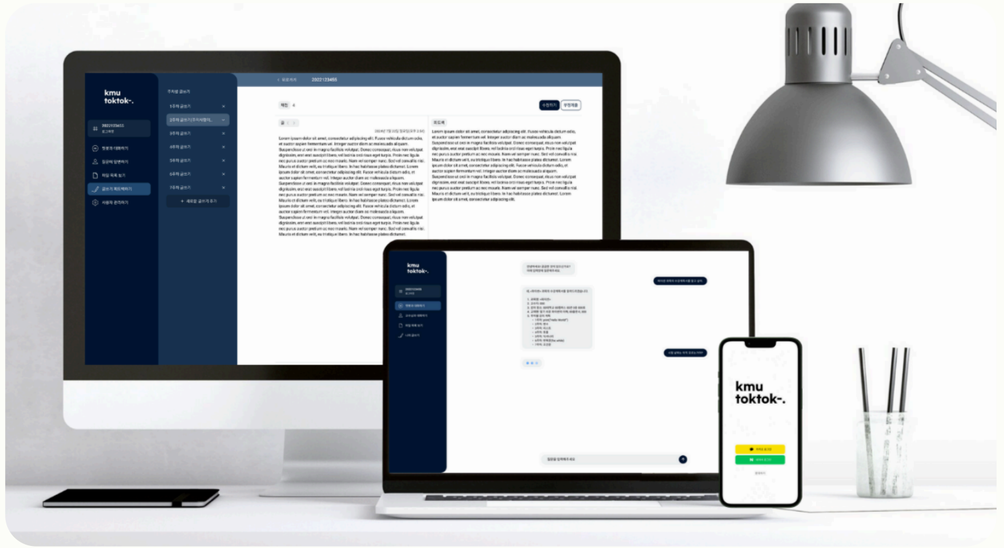
PROJECTS

크무톡톡

2024.03-2024.12, 누적사용자 3161명

기여도 ★★★★★

시스템 설계, 데이터베이스 설계, 요구사항 분석
백엔드 팀장 및 배포, 유지보수 담당



대형 강의에서 발생하는 중복 질문, 과제 지원등의 비효율적인 문제를 해결하기 위해 챗봇이 포함된 AI 기반 학습 관리 시스템(LMS)을 개발했습니다.
이 시스템은 OpenAI Assistant(생성형 AI, GenAI)를 활용하여 반복적인 질문에 자동으로 응답하고, 학생들에게 맞춤형 1:1 피드백을 제공함으로써 강사의 부담을 줄이고 강의 질을 향상시켰습니다.
이 과정에서 자주 발생하는 질문과 즉각적인 응답이 필요한 부분들을 분석하고, 이를 OpenAI 기술로 자동화하여 문제를 해결했습니다.

PROJECTS

크무톡톡

요구사항 분석

배경

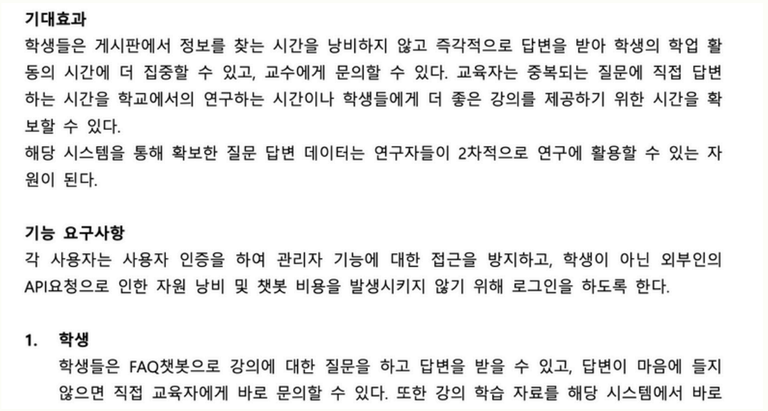
대형 원격 교과목에서 강사들은 반복적인 질문과 과제 관리에 많은 시간을 소비하고 있으며, 학습자들의 학습 만족도는 실시간 피드백의 부족으로 낮아지고 있습니다.

목표 및 주요 요구사항

- 자동화된 피드백 제공: AI를 활용해 중복 질문에 자동 응답
- 과제 및 성적관리: 과제 제출 상태와 성적 계산을 자동화
- 개인화된 학습 피드백: 사용자의 과제를 바탕으로 인공지능 기술을 활용하여 맞춤 피드백 제공
- 확장성: 새로운 요구사항도 반영할 수 있는 안정적이고 확장 가능한 시스템 구축

비기능 요구사항

- 과제 제출이 집중되는 일요일 피크타임에도 안정적이고도 신뢰성 있는 과제 제출을 지원
- 외부 사용자가 학생의 정보에 접근하지 못하도록 접근 제어
- 사용자의 제출 및 요청기록을 log로 서버에 기록



기대효과

학생들은 게시판에서 정보를 찾는 시간을 낭비하지 않고 즉각적으로 답변을 받아 학생의 학업 활동의 시간에 더 집중할 수 있고, 교수에게 문의할 수 있다. 교육자는 중복되는 질문에 직접 답변하는 시간을 학교에서의 연구하는 시간이나 학생들에게 더 좋은 강의를 제공하기 위한 시간을 확보할 수 있다.
해당 시스템을 통해 확보한 질문 답변 데이터는 연구자들이 2차적으로 연구에 활용할 수 있는 자원이 된다.

기능 요구사항

각 사용자는 사용자 인증을 하여 관리자 기능에 대한 접근을 방지하고, 학생이 아닌 외부인의 API요청으로 인한 자원 낭비 및 챗봇 비용을 발생시키지 않기 위해 로그인을 하도록 한다.

1. **학생**
   학생들은 FAQ챗봇으로 강의에 대한 질문을 하고 답변을 받을 수 있고, 답변이 마음에 들지 않으면 직접 교육자에게 바로 문의할 수 있다. 또한 강의 학습 자료를 해당 시스템에서 바로

요구사항 분석 페이지 일부입니다.

PROJECTS

크무톡톡

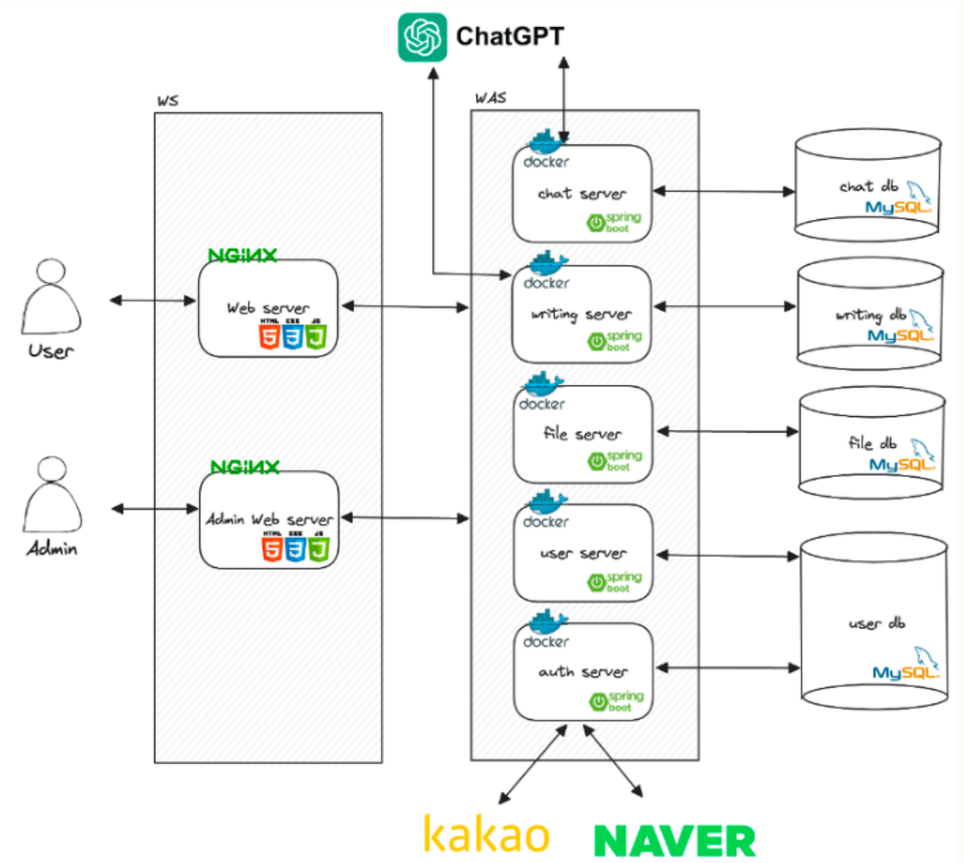
시스템 아키텍처

클라이언트-서버 아키텍처

시스템을 Web Server, Web Application Server, Database로 구성된 3-Tier 구조로 분리함으로써, 다른 부분에 영향을 주지 않고 새로운 기능을 추가하거나 수정할 수 있도록 설계했습니다. 이를 통해 확장성과 유지보수성이 크게 향상되었습니다.

마이크로서비스 아키텍처

애플리케이션을 독립적으로 배포 가능한 작은 서비스 단위로 구성하고, 물리적으로 분리하여 운영했습니다. 이를 통해 특정 서버에서 발생한 문제가 다른 서비스로 전파되는 것을 방지할 수 있었으며, 저렴한 다수의 서버 인스턴스를 활용하고 필요 시 스케일 아웃하여 운영 비용을 크게 절감할 수 있었습니다. (계획한 예산의 50%로 절감)



PROJECTS

크무톡톡

개발

기술스택

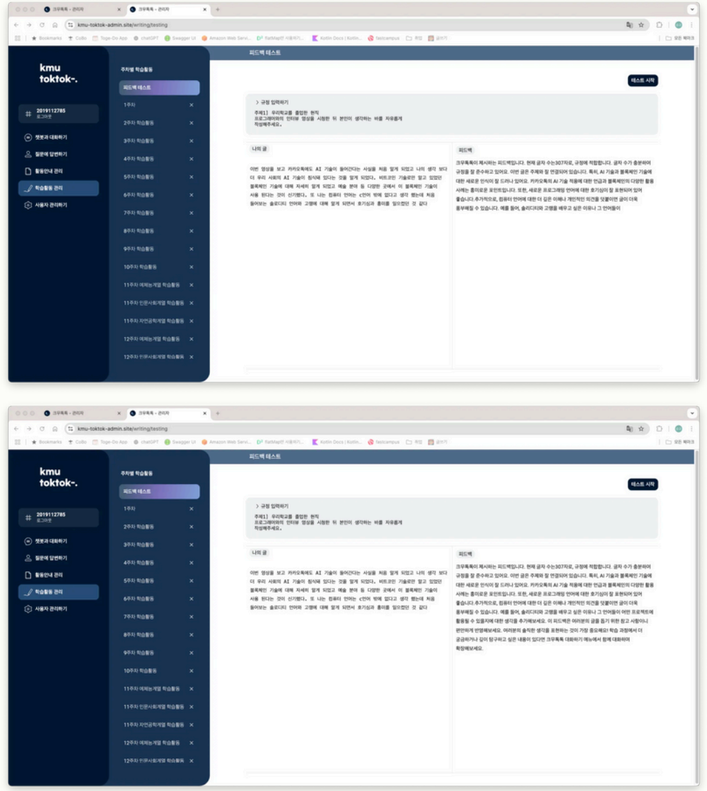
- Kotlin, Spring Boot, Spring Webflux, FastAPI, MySQL
- Spring Security, Jwt, JUnit, Apache Jmeter
- AWS, Nginx, Docker

개발 주요 사항

- Kakao, Naver의 Oauth를 통해 사용자 회원가입 및 로그인
- 인공지능 챗봇 및 학생의 과제 피드백을 위한 OpenAI Assistant 연동
- 파일 업로드 및 다운로드 API 개발
- 학생 피드백 제공 시 빠른 응답을 위해 SSE(Server-Sent Events)를 활용한 실시간 처리 구현

깃허브

- 메인서버: https://github.com/CoBo-Developers/KMU-TOKTOK2
- 피드백 및 SSE Webflux 서버는 private



OpenAI와 SSE를 사용한
실제 피드백 사용 화면

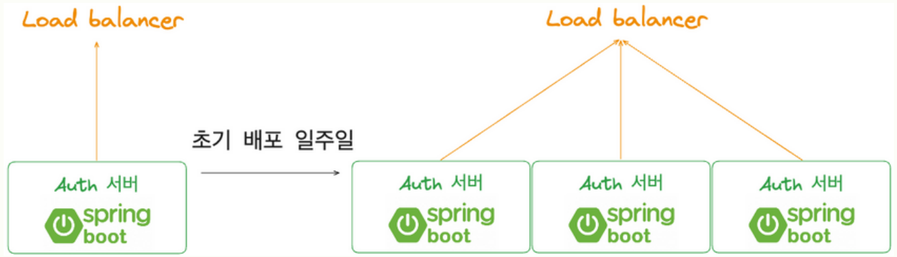
PROJECTS

크무톡톡

개발 포인트

Scale out

기능별로 각각 독립적인 서버로 설계하여, 트래픽이 집중되는 기간에는 인스턴스를
복제하여 Load balancer에 연결해 트래픽 문제를 해결했습니다.



데이터베이스 접근 최소화

JPA를 사용하는 대신 시스템을 분석하여, 조회가 많은 테이블의 경우 insert와 update 시에만 데이터베이스에 접근하도록 설계했습니다.
JVM의 메타스페이스 영역에 데이터를 저장해 조회함으로써 응답 속도를 최대 50%까지 개선하고, 데이터베이스 트래픽을 효과적으로 분산시켰습니다.

비동기 멀티쓰레딩을 활용한 응답속도 개선

코드를 분석하여 의존성이 없는 작업들을 식별하고, 비동기 멀티쓰레딩으로 동시에 실행하도록 설계했습니다.
이를 통해 병렬 처리가 가능해져 사용자의 응답 속도를 효과적으로 개선할 수 있었습니다.

```
CompletableFuture.runAsync {
    feedbackRepository.save(
        Feedback(
            request = studentPostFeedBackReq.content,
            response = feedback,
            studentId = authentication.name
        )
    )
}
```

SSE(ServerSent-Event)를 통해 사용자에게 빠른 응답 제공

SSE를 활용하여 학생 개별 피드백을 ChatGPT처럼 즉각적이고 순차적으로 전달함으로써,
실시간 상호작용을 지원하고 사용자 경험을 크게 향상시켰습니다.

PROJECTS

크무톡톡

결과

3161명
누적 사용자 수

24075회
챗봇 요청 횟수

18234회
피드백 사용 횟수

안정성
JMeter를 활용한 테스트 결과,
0.5 CPU와 0.5GB RAM 환경의 서버에서
OpenAI 최대 처리량인 분당 300개의 피드백을
안정적으로 처리할 수 있음을 확인했습니다.

실제 서비스 환경에서도 클라이언트가 요청한 트래픽을
안정적으로 처리했습니다.

결과 보고서

결과보고서 링크
계명대 시행공문
해당 링크에서 PDF를 확인 할 수 있습니다.

PROJECTS

크무톡톡

느낀점 및 인상깊었던 부분

OpenAI Assistant를 SSE로 연결

기존 방식에서는 생성된 응답이 한 번에 전달되어, 체감되는 응답 속도가 너무 느리다는 사용자 의견이 많았습니다. 이를 해결하기 위해 OpenAI 채팅 응답을 SSE로 변경하는 리팩토링을 진행했습니다. WebFlux의 Flux를 활용하여 응답 데이터를 파싱하고, 필요한 값을 추출해 사용자에게 실시간으로 전달하도록 개선했습니다.

```
ServerResponse
    .ok()
    .contentType(MediaType.TEXT_EVENT_STREAM)
    .body(BodyInserters.fromServerSentEvents(responseStream))
    .awaitSingle()
```

Repository layer를 커스텀

과제 정보는 매주 2~3개씩 추가되지만, 조회는 3,000번 이상 발생합니다. 이러한 특성을 고려하여, JPA 대신 JDBC를 활용해 커스텀 리포지토리를 구현하고 성능을 최적화했습니다. 데이터 저장과 수정은 데이터베이스와 JVM 메타스페이스에 동시에 수행하며, 조회는 메타스페이스에서 처리하여 응답 속도를 절반으로 단축했습니다.
(비록 JPA는 캐시를 지원하지만 직접 만들어서 관리해보고 싶었습니다.)

```
//INSERT
else if(assignment.id == null || !assignmentHashMap.containsKey(assignment.id)) {
    val savedAssignment = this.saveInDB(assignment) //JDBC 사용해 저장
    assignmentHashMap[savedAssignment.id!!] = savedAssignment
    return savedAssignment
}
//UPDATE
else{
    val savedAssignment = this.updateInDB(assignment) //JDBC 사용해 업데이트
    assignmentHashMap[savedAssignment.id!!] = savedAssignment
    return savedAssignment
}
```

PROJECTS

크무톡톡

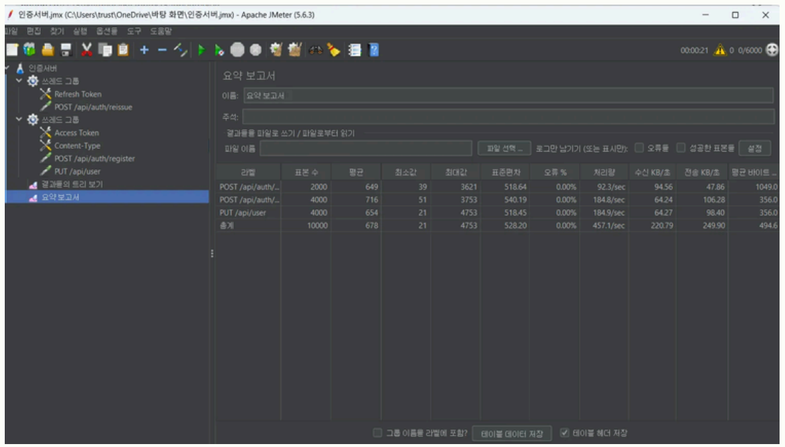
느낀점 및 인상깊었던 부분

Apache Jmeter를 통한 부하 테스트

서비스의 성능을 검증하기 위해 부하 테스트를 진행했습니다.

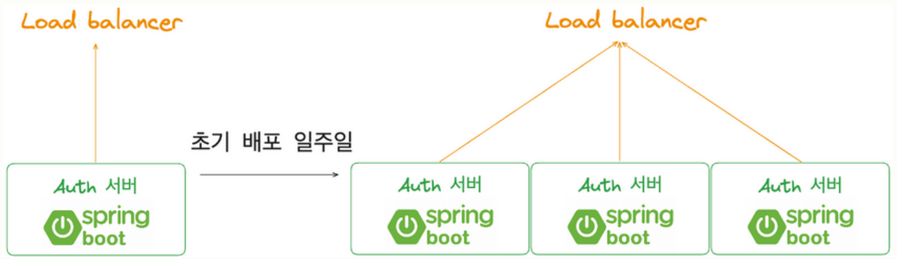
- 스트레스 테스트: 동시 접속자 100명을 대상으로 서비스가 안정적으로 요청을 처리할 수 있는지 확인했습니다.
- 스파이크 테스트: 일요일 과제 제출 시간과 같은 트래픽이 급증하는 상황에서의 성능을 평가했습니다.

클라이언트가 요청한 트래픽을 안정적으로 처리할 수 있는지 검증한 후 결과를 보고했습니다.



Scale out 사용

트래픽 증가로 인한 scale out은 아니었지만, 예상보다 많은 사용자가 SMTP 기능을 이용하면서 일일 예상 메일 요청 500회를 초과하는 문제가 발생했습니다.
이를 해결하기 위해 SMTP 서버를 2개 추가로 복제하고,
로드 밸런서를 통해 부하를 1:1:1로 분산하도록 구성했습니다.
그 결과, 일일 최대 메일 요청 처리량을 1500회까지 확장하여 안정적인 서비스 운영이 가능하도록 개선했습니다.



PROJECTS

TogeDo

2024.12 - 진행 중

기여도 ★★★★★

시스템 설계, 데이터베이스 설계
백엔드 개발 및 배포



새롭게 공부하는 기술들을 실제로 적용해보기 위해, 자유롭게 기술들을 적용해 보고 꾸준히 유지보수 할 앱 플랫폼을 개발했습니다.
이 플랫폼은 소셜 로그인을 활용해 간편하게 회원가입 및 로그인을 할 수 있으며, 개인 일정과 그룹 일정을 통합하여 관리하도록 합니다.
또한, 사용자 간 원활한 소통을 위해 실시간 알림(SSE, 앱 푸쉬) 및 채팅 서비스를 구축하여 사용자 경험을 극대화했습니다.

PROJECTS

TogeDo

요구사항 분석

배경

기존에는 개인 일정만 관리 할 수 있는 플랫폼은 많았지만, 친구들과 일정을 공유하고 조율할 수 있는 플랫폼은 부족하다고 느꼈습니다.
이를 해결하기 위해, 친구 및 그룹 간 일정을 쉽게 공유하고 조율할 수 있는 플랫폼을 개발하게 되었습니다.

목표 및 주요 요구사항

- 소셜 로그인: 보안과 편리를 위해 카카오와 구글을 이용한 회원가입과 로그인
- 일정 공유: 사용자의 개인 일정을 불러와 같은 그룹의 친구와 일정을 조율
- 실시간 알림: 친구 초대 및 그룹 초대와 같은 알림을 앱 실행 중 여부와 상관없이 실시간으로 알림
- 그룹 채팅: 같은 그룹에 속한 사용자끼리 일정을 조율하기 위해 채팅을 구현

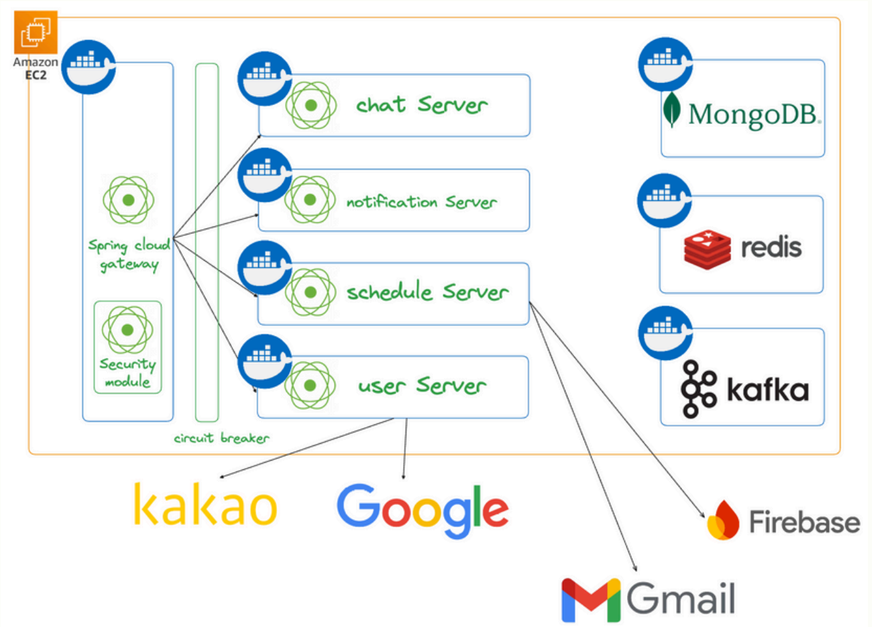
비기능 요구사항

- 생성 및 삭제 중 에러가 발생하더라도, Saga패턴으로 장기 트랜잭션에서도 데이터의 무결성을 지원(현재 리펙토링 진행 중입니다.)
- 본인의 일정을 조회할 경우, 인증된 사용자만 접근하도록 권한 검사를 수행

PROJECTS

TogeDo

시스템 아키텍처



Kafka 활용 마이크로서비스 아키텍처

대용량 트래픽을 대비해 Kafka를 활용하여 비동기적으로 데이터를 처리하고, 확장성 있는 시스템을 구축했습니다. API Gateway를 통해 서비스별 독립적인 기능 개발과 업데이트가 가능하며, Redis를 활용해 빠른 데이터 조회 성능을 제공하여 전체 시스템의 효율성을 극대화했습니다.

CI/CD

Jenkins를 활용하여 CI/CD 파이프라인을 구축하였습니다. 이를 통해 코드 변경 사항이 자동으로 빌드, 테스트, 배포 단계까지 이어지도록 설정하여 개발 효율성을 높였습니다. 또한 각 스프린트마다 테스트 코드를 추가하여 통합되는 코드들을 지속적으로 검증 할 수 있는 환경을 마련했습니다.
(서버의 비용 문제로 현재는 Github action을 통한 CI/CD 환경으로 변경했습니다.)

Heaxgonal Architecture

멀티모듈을 활용해 각 layer가 port, adapter로 통신하며 domain의 외부 의존성을 낮춘 Heaxgonal Architecture를 사용했습니다.
이를 통해 리펙토링 과정에서 다른 Layer의 변경없이, 의존성을 낮추어 수정과 확장이 가능하게 설계했습니다.
(현재 리펙토링 진행 중입니다.)

PROJECTS

TogeDo

개발

기술스택

- Kotlin, Spring Webflux, MongoDB
- Spring Gateway, Spring CircuitBreaker, Stomp, Jwt, JUnit, Locust
- Redis, Kafka, Debezium, AWS, Docker

개발 주요 사항

- Spring Gateway, Spring CircuirtBreaker, Kafka를 사용하는 MSA 환경 구축
- Gateway에 Jwt 인증을 확인하는 filter 추가
- 사용자가 앱에 접속하면 SSE를 통한 실시간 알림, 접속하지 않으면 Device token을 활용한 앱 푸쉬알림
- Stomp 라이브러리를 활용한, 실시간 그룹 채팅

깃허브

- 백엔드서버: https://github.com/Seungkyu-Han/Toge-do-backend

PROJECTS

TogeDo

개발 포인트

관심사 분리

모든 서버는 해당 서버의 고유 기능과 관련된 작업만 처리하며, 관심사가 다른 작업을 수행해야 하는 경우 Kafka를 통해 담당 서버로 이벤트를 전송해서 관심사에 맞는 서버가 처리하도록 했습니다. 이를 통해 시스템의 확장성과 유지보수성을 강화했습니다.

Redis 캐싱

사용자가 반복적으로 조회하는 데이터를 효율적으로 처리하기 위해, persistence layer에 Redis를 도입했습니다. 커스텀 Repository에서 데이터를 Redis에서 조회하고, 값이 존재하지 않으면 MongoDB에서 데이터를 가져오도록 구성하였습니다. 조회, 정보 수정 및 추가 작업 시 해당 데이터를 Redis에 일정 시간 동안 저장하며 캐싱을 관리했습니다. 이를 통해 데이터베이스의 부하를 줄이고, 사용자의 체감 속도를 향상시켰습니다.

Spring Gateway filter를 통한 Jwt 인증

각 서버에서 개별적으로 JWT 인증을 수행하는 방식은 관심사의 분리가 부족하다고 판단하였습니다. 이를 해결하기 위해 Gateway의 Filter에서 JWT 인증을 중앙화하여 처리하도록 설계하였습니다. Gateway에서 JWT를 검증하고 사용자의 고유 정보를 추가하여 각 서버로 요청을 전달했습니다.

MSA 환경에서 CircuitBreaker 적용

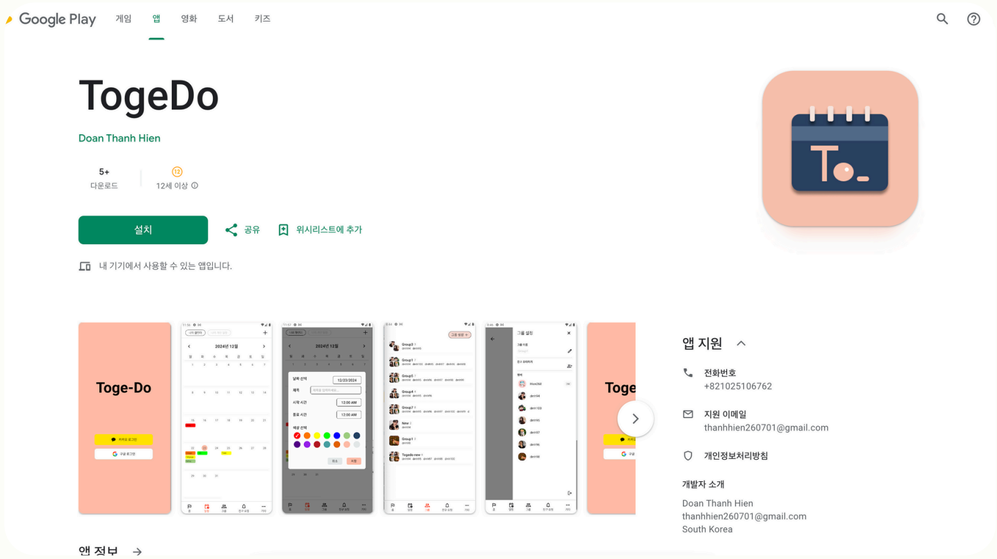
MSA 환경에서는 한 서버에서 발생한 장애가 다른 서버로 전파되지 않도록 설계하는 것이 중요합니다. 이를 위해 서버 간 연결 구간에 CircuitBreaker를 적용하여 장애 전파를 효과적으로 차단하였습니다. 특정 요청에서 반복적으로 Time-out이 발생하는 경우, CircuitBreaker가 해당 서버로의 접근을 일시적으로 차단하여 시스템의 안정성을 유지할 수 있도록 구현하였습니다.

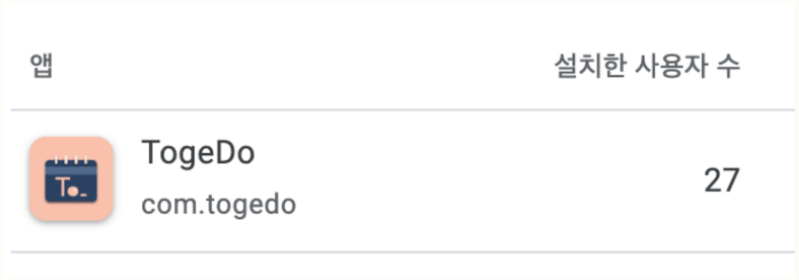
PROJECTS

TogeDo

결과

플레이스토어 출시
안드로이드 플레이스토어에서 서비스 중입니다.
지속적인 기능 추가와 성능 개선, 버그 수정을 이어나갈 계획입니다.





PROJECTS

TogeDo

느낀점 및 인상깊었던 부분

Transaction 적용

이번 프로젝트에서는 @Transactional이 아닌 다른 방식으로 트랜잭션을 구현해 보았습니다. MongoDB의 TransactionalOperator를 활용하여 특정 로직에만 선택적으로 트랜잭션을 적용했습니다. 이를 통해 트랜잭션 범위를 세밀하게 제어하고, 성능을 최적화할 수 있었습니다.

```
transactionalOperator.executeAndAwait {
    receiverDocument = friendService.approveFriend(id, friendId).awaitSingle()
    senderDocument = friendService.addFriend(friendId, id).awaitSingle()
}
```

자세한 내용은 블로그에 작성되어 있습니다.
https://seungkyu.tistory.com/51

MongoDB를 사용한, MSA 환경 채팅서버 구현

MSA에서 채팅은 내용을 저장하고 Kafka에 이벤트를 전송해 연결된 웹 소켓에 채팅을 전송하기에, 외부 리소스인 MongoDB와 Kafka를 두 번 호출해야 하는 비효율성이 있었습니다. 이를 MongoDB의 change stream 기능을 활용하여, MongoDB에 채팅을 저장하는 동시에 실시간으로 웹 소켓에 채팅을 전송할 수 있도록 구성하여 외부 리소스를 두 번 호출하지 않고, MongoDB만으로 효율적으로 채팅 전송을 처리할 수 있었습니다.

```
reactiveMongoTemplate.changeStream(ChatDocument::class.java)
    .listen()
    .doOnNext{
        item →
        val chatDocument = item.body
        val operationType = item.operationType

        if(chatDocument ≠ null && operationType == OperationType.INSERT) {
            simpMessageSendingOperations.convertAndSend(
                destination: "/sub/${chatDocument.groupId}", messageDocumentToDto(chatDocument)
            )
        }
    }.subscribe()
```

자세한 내용은 블로그에 작성되어 있습니다.
https://seungkyu.tistory.com/54

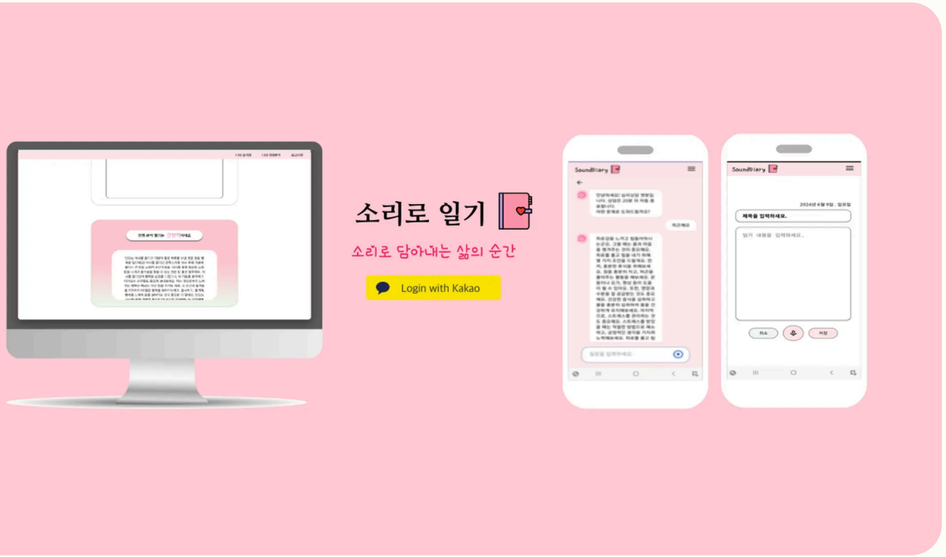
PROJECTS

소리로 일기

2024.03 - 2024.07

기여도 ★★★★☆

PM
데이터베이스 설계, 시스템 설계
백엔드 설계 및 개발, 배포



소리로 일기 프로젝트는 경북대학교 산학협력 프로젝트의 일환으로, 음성을 통해 감정을 분석하고 일기를 작성하는 웹 애플리케이션입니다.
우울증과 불안장애 같은 정신건강 문제를 완화하고, 사생활 노출에 대한 거부감을 줄이는 것을 목표로 하고 있습니다.
사용자는 음성을 통해 일기를 작성할 수 있으며, 작성된 일기를 바탕으로 감정 분석과 AI 상담 기능을 제공합니다.

PROJECTS

소리로 일기

요구사항 분석

배경

현대 사회에서 정신건강 문제는 중요한 이슈로 대두되고 있지만, 많은 사용자들은 정신건강 서비스를 이용하는 데 부담을 느끼고 있습니다. 이에 따라, 접근성이 높고 부담 없이 이용할 수 있는 정신건강 서비스 애플리케이션을 개발하게 되었습니다.

목표 및 주요 요구사항

- 일기를 바탕으로 감정분석: 사용자가 작성한 일기를 Clova 감정분석 API를 통해 감정을 분석
- 일기와 채팅을 활용한 상담 제공: ChatGPT로 작성된 일기와 상담 내용을 바탕으로 사용자 맞춤형 상담 서비스를 제공
- 기존 상담 내역 조회: 과거 작성된 일기, 감정분석 결과, 상담 내역을 한눈에 조회할 수 있도록 지원

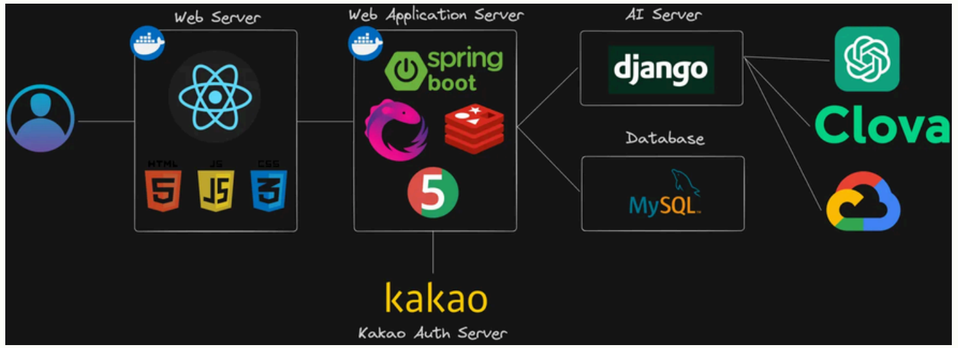
비기능 요구사항

- 높은 접근성을 보장하기 위해 인공지능 기능을 제외한 모든 요청은 1초 이내에 응답
- 새벽과도 같은 모든 시간대에 사용 가능

PROJECTS

소리로 일기

시스템 아키텍처



모듈화된 구조

Web Server, Web Application Server, AI Server의 역할을 분리하여 설계하였으며, AI Server는 인공지능 리소스 활용에 따른 잦은 변경에 대비해 독립적인 모듈로 개발되었습니다. Web Application Server는 AI Server로 HTTP 요청을 보내는 방식으로 통합을 구현했습니다.

멀티쓰레드 활용

이 프로젝트에서는 처음으로 멀티쓰레드를 활용하여 개발을 진행했습니다. 인공지능 관련 API 호출은 응답 시간이 5초 이상 소요되는 경우가 많아, 이 시간을 활용하여 다른 작업을 처리할 수 있도록 했습니다. 이를 위해 CompletableFuture를 사용하여 비동기적으로 쓰레드를 추가로 생성하고, 인공지능 API 응답 대기 중에도 데이터베이스 저장 등의 작업을 동시에 처리할 수 있게 했습니다. 이를 통해 시스템의 효율성을 높이고, 응답 시간을 최적화할 수 있었습니다.

PROJECTS

소리로 일기

개발

기술스택

- Kotlin, Spring Boot, MySQL
- ReactiveX, Jwt, Redis, JUnit5, Apache Jmeter
- AWS, Docker, ChatGPT, Clova

개발 주요 사항

- 사용자가 작성한 일기를 기반으로 Clova API를 호출하여 감정 분석 기능 구현
- JUnit5를 활용한 Spring 애플리케이션 테스트 자동화로 안정성 확보
- HTTP 호출 과정에서 보안 강화를 위해 서버 HTTPS 암호화 설정 및 헤더 관리

깃허브

- 통합 서버: https://github.com/Seungkyu-Han/CapstoneDesign2

PROJECTS

소리로 일기

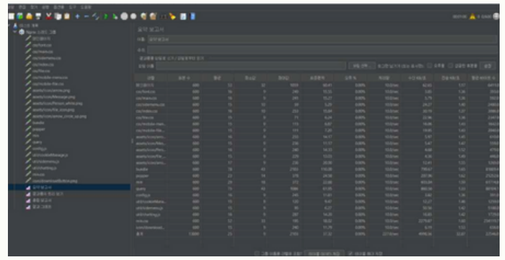
개발 포인트

QueryDSL 활용

JPA와 JPQL만으로는 동적인 쿼리문을 작성하는 데 한계가 있어, QueryDSL을 활용하여 동적 쿼리 생성을 구현했습니다.
이를 통해 조건에 따라 유연하게 쿼리를 생성하고, 코드의 가독성과 유지보수성을 높일 수 있었습니다. QueryDSL을 사용함으로써 복잡한 쿼리 작업도 더욱 직관적이고 효율적으로 처리할 수 있었습니다.

Apache Jmeter를 사용해 서버의 부하 테스트

배포한 서버에서 수용 가능한 최대 사용자의 수를 알아보기 위해 Apache Jmeter로 부하 테스트를 수행했습니다.
사용 기반으로 시나리오를 설계하고 안정성을 측정해, 서버 성능을 최적화하는 데 도움을 줄 수 있었습니다.



PROJECTS

소리로 일기

결과

소프트웨어 저작권 등록



제 C-2024-020488 호

저작권 등록증

1. 저작물의 제호(명칭)	소리로 일기(STT Diary)
2. 저작물의 종류	컴퓨터프로그램저작물>응용프로그램>컨텐츠 개발용 S/W>Web 저작도구
3. 저작자 성명(법인명)	경북대학교산학협력단 대구광역시 북구 대학로 80 글로벌플라자 804호 기술사업팀
4. 생년월일(법인등록번호)	176271-0001921
5. 창작연월일	2024년06월05일
6. 공표연월일	-
7. 등록연월일	2024년06월14일
8. 등록사항	저작자 : 경북대학교산학협력단, 창작 : 2024.06.05

『저작권법』 제53조에 따라 위와 같이 등록되었음을 증명합니다.

2024년 06월 14일

한국저작권위원회

결과 보고서

결과보고서 링크
해당 링크에서 PDF를 확인 할 수 있습니다.

PROJECTS

소리로 일기

느낀점 및 인상깊었던 부분

SpringBoot에서 외부 API 호출
백엔드 서버에서 외부 서버의 API 호출 경험이 많지 않았고,
저만의 convention을 가지고 있지도 않았습니다. 필요한 경우에만
구글에서 검색해가며 해결했는데, 이번에 RestTemplate을
사용해보며 원하는 외부 서버(OpenAI, Naver)에서 자유롭게 API를
호출하여 정보를 가져올 수 있게 되었습니다.
(현재는 RestTemplate이 아닌, WebClient를 사용합니다.)

```
val restTemplate = RestTemplate()

val httpHeaders = HttpHeaders()
httpHeaders.add("Content-Type", "application/x-www-form-urlencoded;charset=utf-8")
httpHeaders.add("Authorization", "Bearer $token")

return restTemplate.exchange(
    RequestEntity<Any>(httpHeaders, HttpMethod.POST, URI.create(jsonReqUrl)),
    AuthKakaoInfoRes::class.java
).body
```

노션을 사용해 협업과 애자일에서의 스프린트
현업에서 일하시는 멘토님과 함께 개발을 진행하며 협업과 애자일 방식
에 대해 많은 것을 배울 수 있었습니다. 멘토님은 개발 과정에서 짧고
많은 회의를 추천하셨고, 스프린트를 진행하면서 코드 리뷰와 테스트
코드의 중요성을 강조하셨습니다. 이러한 조언 덕분에 프로젝트 전반에
걸쳐 원활하게 커뮤니케이션하며, 마지막까지 큰 문제 없이 프로젝트를
성공적으로 마칠 수 있었습니다.



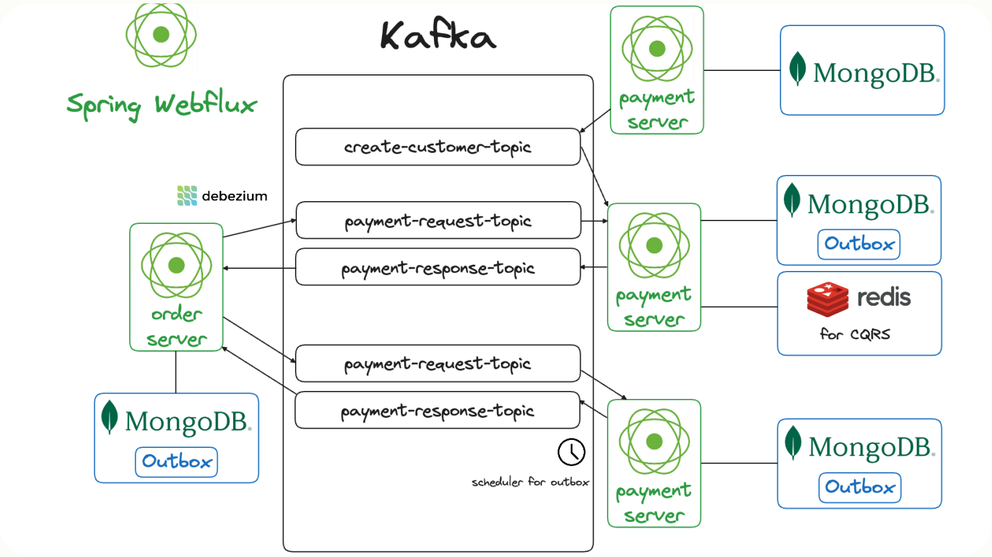
PROJECTS

MSA 프로젝트

2024.03 - 2024.07

기여도 ★★★★★

시스템 설계, 백엔드 개발



Clean Architecture를 위해 DDD, Hexagonal Architecture를 연습해보고
MSA에 많이 사용되는 패턴인 Saga, CQRS, Outbox, CDC 패턴을 적용해보기 위해 개인적으로 진행한 서브 프로젝트입니다.
카카오 기술 블로그: https://tech.kakao.com/posts/632
우아한형제들 테크 영상: https://www.youtube.com/watch?v=BnS6343GTkY
해당 기술들을 참고해 개인적으로 프로젝트를 진행했습니다.

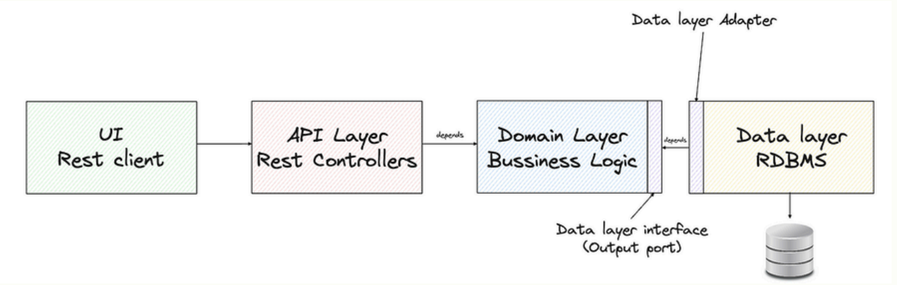
PROJECTS

MSA 프로젝트

시스템 아키텍처

Hexagonal Architecture

domain 위주로 개발을 진행하면서도, 해당 domain 자체가 외부 모듈의 변경에 영향을 받지 않도록 domain을 중심으로 port와 adapter를 사용하는 Hexagonal Architecture를 사용했습니다.
블로그 포스팅: https://seungkyu.tistory.com/64



멀티모듈

Hexagonal Architecture를 적용하는 과정에서 동일한 코드와 반복되는 패턴이 여러 곳에서 발생하는 것을 확인했습니다. 이를 효율적으로 관리하고 재사용성을 극대화하기 위해, 기능별로 모듈을 분리하고 공통적인 부분을 공유할 수 있도록 멀티모듈 구조로 개발했습니다. 이러한 방식으로 코드의 중복을 최소화하고 유지보수성을 높이는 동시에, 각 모듈이 독립적으로 동작할 수 있도록 설계하여 확장성과 유연성을 강화했습니다.



PROJECTS

MSA 프로젝트

개발

기술스택

- Kotlin, Spring Webflux, MongoDB
- Coroutine, Redis
- Docker, Kafka, Kafka-ui, Avro, Debezium

개발 주요 사항

- MSA 환경에서 Kafka 이벤트 메시지의 안정적인 장기 트랜잭션 관리

깃허브

- https://github.com/Seungkyu-Han/micro_service_webflux

PROJECTS

MSA 프로젝트

개발 포인트

Saga 패턴

Saga 패턴을 적용하여 여러 서버에 걸친 요청과 응답을 트랜잭션으로 동작하도록 설계했습니다. 각 메시지의 Request와 Response를 기반으로 해당 서버에서의 이벤트 처리 상태를 확인하고, 이를 바탕으로 process 또는 rollback을 결정하도록 구현했습니다. 이를 통해 장기 트랜잭션을 여러 개의 단기 트랜잭션으로 분리하여 관리할 수 있도록 하였으며, 특히 결제 서버의 결제 상태나 업체 서버의 운영 상태에 따라 전체 프로세스를 롤백할 수 있도록 구성하여 결제 흐름을 트랜잭션적으로 보장했습니다.
블로그 포스팅: https://seungkyu.tistory.com/65

Outbox 패턴

Saga 패턴을 적용하는 과정에서 이벤트 발행과 상태 저장 중 에러가 발생하면, 발행되지 말아야 할 이벤트가 잘못 전송되는 문제가 발생할 수 있습니다. 이를 방지하기 위해 Outbox 패턴을 활용했습니다. 이벤트를 먼저 Outbox 테이블에 저장한 후, Scheduler를 사용하여 아직 발행되지 않은 이벤트를 Kafka를 통해 전송하도록 설계했습니다. 이를 통해 메시지 발행과 상태 저장을 하나의 트랜잭션으로 관리할 수 있도록 구현했습니다.
블로그 포스팅: https://seungkyu.tistory.com/67

PROJECTS

MSA 프로젝트

개발 포인트

CDC 패턴

Outbox 패턴을 적용하기 위해 스케줄러를 사용했지만, 빠른 시간 내에 동기화가 이루어지지 않는 문제가 발생했습니다. 또한, 스케줄러 실행 시 해당 시점에만 CPU 사용량이 급증하고, 서버가 Outbox 데이터를 주기적으로 조회해야 하는 부담이 있었습니다. 이를 해결하기 위해 CDC(Change Data Capture) 패턴을 적용했습니다. Debezium을 사용해 데이터베이스의 트랜잭션을 실시간으로 감지하고, 발생하는 이벤트를 즉시 Kafka를 통해 전송하도록 구현했습니다. 이를 통해 Outbox에 저장된 이벤트가 빠르게 발행될 수 있었고, 서버에서 데이터베이스를 조회하는 횟수를 줄일 수 있었습니다.
블로그 포스팅: https://seungkyu.tistory.com/71

CQRS 패턴

Customer 서버에서 사용자 정보는 생성되지만, 이를 직접 조회하지 않고 Payment 서버 등 다른 서버에서 조회해야 하는 상황이 있었습니다. 이를 해결하기 위해 Customer 서버에서 Create, Update, Delete 연산이 발생할 때 Kafka를 통해 이벤트를 전송하고, 사용자의 정보를 필요로 하는 서버에서는 해당 정보를 Redis에 저장하여 빠르게 조회할 수 있도록 CQRS 패턴을 적용했습니다. 이 방식으로 데이터의 일관성을 유지하면서도 조회 성능을 최적화할 수 있었습니다.
블로그 포스팅: https://seungkyu.tistory.com/68

PROJECTS

MSA 프로젝트

느낀점 및 인상깊었던 부분

MSA 환경에서의 장기 트랜잭션 적용

기존에는 비동기 이벤트에서 트랜잭션을 처리할 수 없어서, 트랜잭션이 필요한 경우 Kafka 대신 서버 간 HTTP 통신을 통해 문제를 해결했습니다. 이제는 Saga 패턴을 통해 비동기 이벤트의 장기 트랜잭션을 적용할 수 있게 되어, 현재 준비 중인 Toge-do 어플리케이션에도 해당 패턴을 적용하여 사용자에게 더 빠른 응답 속도를 제공하고, 전체 서비스의 성능을 최적화하려고 합니다.

```
interface SagaStep<T> {  Seungkyu-Han

    fun process(data: T): Mono<Void>  Seungkyu-Han
    fun rollback(data: T): Mono<Void>  Seungkyu-Han
}
```

CQRS를 통한 조회 속도 개선

고객 정보는 고객 서버에서 생성되지만, 해당 정보를 고객 서버에서 직접 조회하지 않습니다. 외부 서버에서 고객 서버로 요청을 보내 정보를 조회해야 하지만, 이를 해결하기 위해 CQRS 패턴을 적용했습니다.
고객 정보의 변동이 발생하면 해당 정보를 이벤트로 전송하고, 각 서버는 이 이벤트를 수신하여 Redis에 데이터를 저장합니다. 이렇게 저장된 정보를 빠르게 조회하여 서비스를 처리할 수 있도록 구현했습니다.

```
@KafkaListener(id = "customer_consumer_group",  Seungkyu-Han
    topics = ["\${kafka.topic.customer-create}"])
fun receive(
    @Payload values: List<String>,
    @Header(KafkaHeaders.RECEIVED_KEY) keys: List<String>,
    @Header(KafkaHeaders.RECEIVED_PARTITION) partitions: List<Int>,
    @Header(KafkaHeaders.OFFSET) offsets: List<Long>
) {
    values.forEach{
        customerCreateMessageListener.createCredit(
            Credit(
                customerId = CustomerId(id = ObjectId(it)),
                totalCreditAmount = Money.ZERO
            )
        ).subscribe()
    }
}
```

한 승 규

Email

trust1204@gmail.com

Github

https://github.com/Seungkyu-Han

Phone

+82 10-4081-9491

Blog

https://seungkyu.tistory.com/
https://seungkyu-han.tistory.com/

ver 2025.03.09